

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 3 — DEBUGGING & OPTIMISATION TASK

➤ YAVANIKA S

[192521258]

Q1. Overflow in Signed Integer Addition (2's Complement)

► Problem Given

- Program adds two signed numbers using 2's complement.
- Wrong result comes for large values.
- Example given: +120 and +50 in 8-bit representation.

Debugging the Issue

- 8-bit signed range is -128 to +127.
- +120 in binary = 0111 1000
- +50 in binary = 0011 0010
- Adding both: $0111\ 1000 + 0011\ 0010 = 1010\ 1010$
- Result 1010 1010 is a negative number in 2's complement.
- But $+120 + 50 = +170$, which is a positive number.
- This wrong sign shows overflow has occurred.
- Reason: correct answer +170 is outside the range +127.
- Two positive numbers added and gave a negative result.
- This is the clear sign of overflow.

Overflow Detection Rule

- Rule 1: If two positive numbers are added and result is negative, overflow happens.
- Rule 2: If two negative numbers are added and result is positive, overflow happens.

- Rule 3: Adding a positive and a negative number never causes overflow.
- Hardware rule: Compare Carry into MSB (C_{in}) and Carry out of MSB (C_{out}).
- If C_{in} is not equal to C_{out} , overflow flag is set to 1.

► Proposed Solution

- Use an overflow flag (V flag) in ALU.
- Set $V = C_{in} \text{ XOR } C_{out}$ after every addition.
- If $V = 1$, stop and report overflow error.
- Use higher bit-width (like 16-bit) if bigger numbers are expected.
- Always check sign bits of operands and result before trusting output.

Flowchart

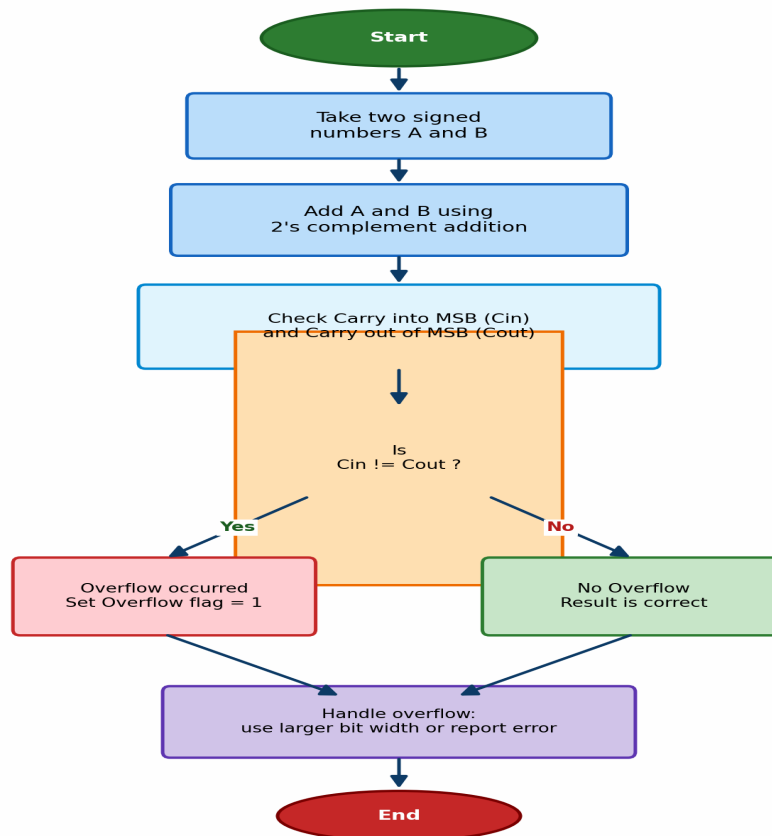


Fig 1.1: Overflow Detection Flowchart

• Q2. Ripple Carry Adder Delay vs Carry Look-Ahead Adder

► Problem Given

- Processor uses a ripple carry adder (RCA).
- Real-time system shows high delay during arithmetic operations.

Debugging the Bottleneck

- In RCA, each full adder waits for carry from previous stage.
- Bit 0 must finish before Bit 1 can start.
- Bit 1 must finish before Bit 2 can start, and so on.
- This is called carry propagation delay.
- Total delay = number of bits x delay of one full adder.
- For large bit-length (32-bit, 64-bit), this delay becomes very high.
- This is why real-time system becomes slow.

Optimized Solution: Carry Look-Ahead Adder (CLA)

- CLA calculates all carries in advance.
- It uses two signals: Generate (G) and Propagate (P).
- $G_i = A_i \text{ AND } B_i$ (generates a new carry).
- $P_i = A_i \text{ XOR } B_i$ (propagates existing carry).
- Carry equation: $C_{i+1} = G_i + (P_i \cdot C_i)$
- All carry bits are found using extra logic gates, not by waiting.
- Carries for all bit positions are ready almost at the same time.

How Delay is Reduced

- RCA delay grows in direct proportion to number of bits (linear delay).
- CLA delay stays almost constant, does not grow much with more bits.
- Extra hardware (gates) is used, but speed improves a lot.
- CLA is best choice for high speed processors and real-time systems.

Flowchart

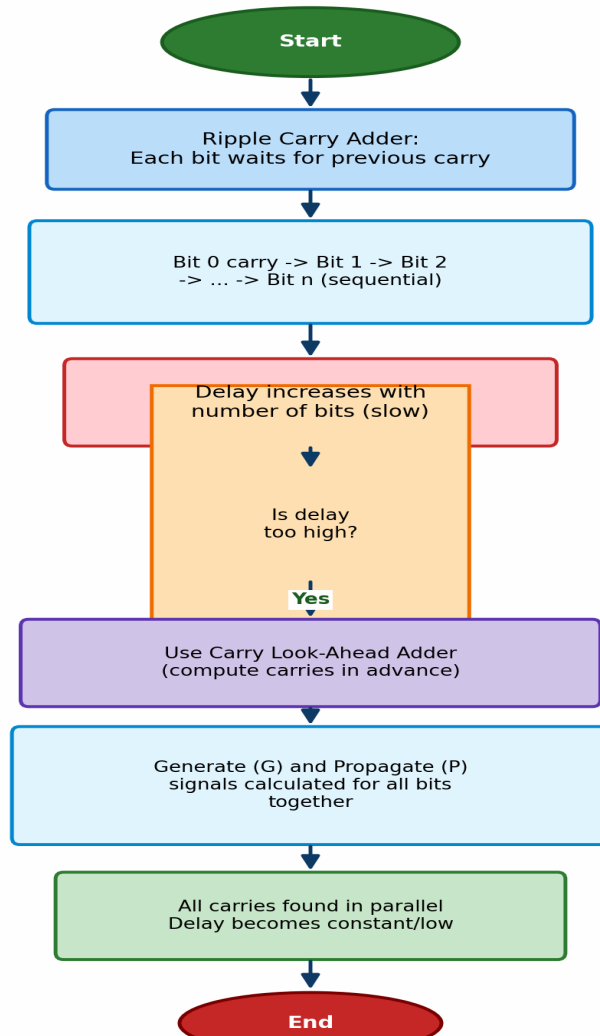


Fig 2.1: Ripple Carry vs Carry Look-Ahead Adder Flowchart

• Q3. Booth's Algorithm - Error in Negative Operands

► Problem Given

- Multiplication unit uses Booth's algorithm.
- Wrong results appear for certain negative operands.

Debugging Step by Step

- Booth's algorithm uses registers A, Q, Q-1 and M.
- A = Accumulator (starts at 0)
- Q = Multiplier, Q-1 = extra bit (starts at 0), M = Multiplicand
- Algorithm checks bit-pair (Q_n, Q-1) at every step.
- If pair is 00 or 11: no arithmetic operation, only shift.
- If pair is 01: add multiplicand ($A = A + M$)
- If pair is 10: subtract multiplicand ($A = A - M$)
- After operation, an arithmetic right shift is done on A, Q, Q-1 together.

Possible Source of Error

- Error 1: Using logical shift instead of arithmetic shift.
- Logical shift loses the sign bit, so negative numbers become wrong.
- Error 2: Missing sign extension when A register is negative.
- Error 3: Wrong number of shift iterations (must equal number of bits, n).
- These mistakes give wrong result mainly when operands are negative.

Correction / Suggested Fix

- Always use arithmetic right shift (sign bit is copied, not lost).
- Correctly sign-extend the Accumulator (A) before shifting.
- Repeat the process exactly n times, where n = number of bits.
- Check bit-pair carefully at every step, do not skip Q-1 bit.
- Test algorithm with a small negative example to confirm correction.

Flowchart

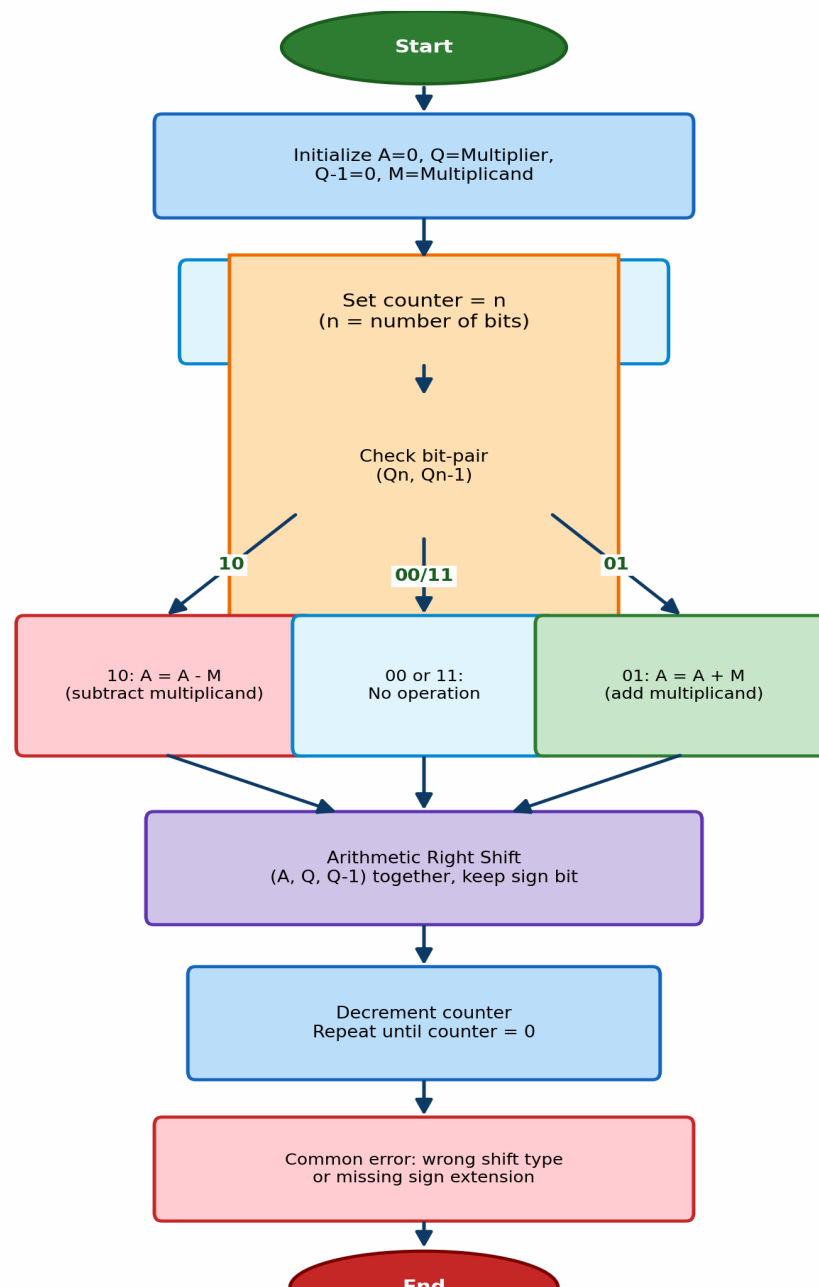


Fig 3.1: Booth's Algorithm Flowchart

• Q4. Restoring Division Inefficiency

► Problem Given

- Embedded system uses restoring division algorithm.
- Repeated restoration steps cause more execution time.

Debugging the Workflow

- In restoring division, remainder is subtracted by divisor at each step.
- If result becomes negative, an extra step is done: add divisor back.
- This 'add back' step is called restoring.
- Restoring step does not help find the answer directly.
- It is only used to undo the wrong subtraction.
- This wastes one extra clock cycle at every negative step.
- For n-bit division, many restore steps can occur.
- This is the unnecessary operation causing inefficiency.

Optimized Approach: Non-Restoring Division

- Non-restoring division removes the separate restore step.
- If previous remainder is positive, next step subtracts divisor.
- If previous remainder is negative, next step adds divisor (instead of restore + subtract).
- Only one correction is done at the last step, if needed.
- Quotient bit is set to 1 if remainder is positive, 0 if negative.

How Performance Improves

- Restoring division needs up to 2 operations per step (subtract + restore).
- Non-restoring division needs only 1 operation per step (add or subtract).
- This nearly cuts the number of operations to half.
- Execution time reduces, which is very useful for embedded systems.

Flowchart

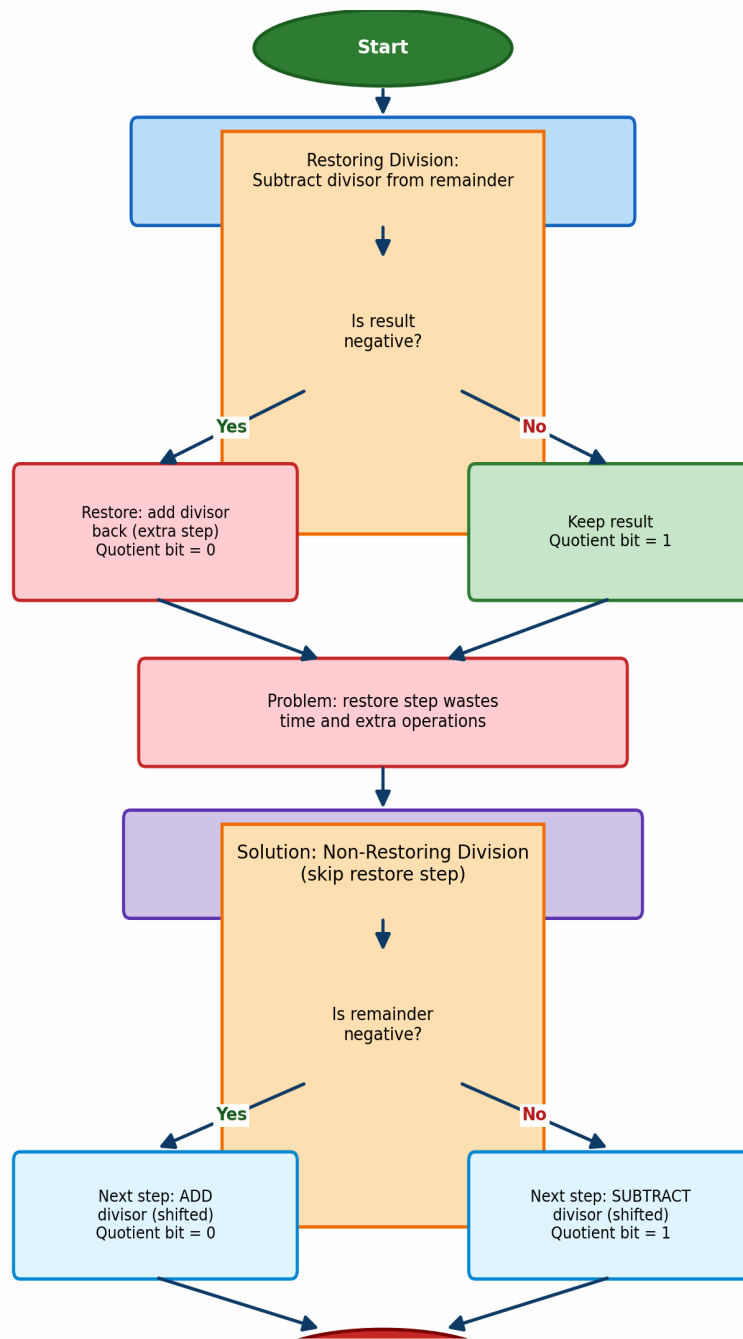


Fig 4.1: Restoring vs Non-Restoring Division Flowchart

• Q5. IEEE 754 Floating Point Addition Error

► Problem Given

- Scientific application uses IEEE 754 single precision format.
- Inaccurate result comes when adding a very small and a very large number.

Debugging the Issue

- IEEE 754 format has sign bit, exponent bits, and mantissa bits.
- Before addition, exponents of both numbers must be made equal.
- This is called exponent alignment.
- The number with smaller exponent shifts its mantissa to the right.
- If the gap between exponents is very large, many mantissa bits shift out.
- Shifted-out bits are lost, this is called precision loss.
- Small number may become almost zero after this shifting.
- After addition, result is normalized again.
- Normalization and rounding can add further small errors.

Root Cause

- Single precision has only 23 mantissa bits, which is limited space.
- Very large exponent difference does not fit in these limited bits.
- So very small number's value gets lost during alignment.

Suggested Optimization Techniques

- Use double precision (64-bit) format, since it has 52 mantissa bits.
- More mantissa bits reduce the chance of losing small values.
- Use algorithmic adjustment: add small numbers together first.
- Then add the combined small sum to the large number at the end.
- This method is known as compensated summation (Kahan summation idea).
- Sorting numbers before adding also helps reduce rounding error.

Flowchart

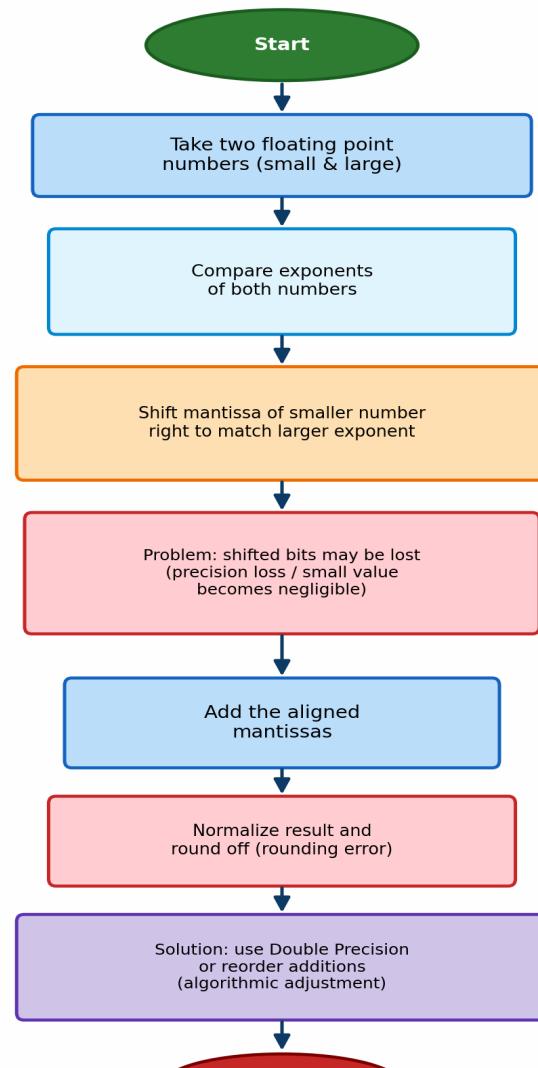


Fig 5.1: IEEE 754 Floating Point Addition Flowchart